

Rapport DevSecOps

Analyse SAST & DAST — Application SecNotes

Pipeline CI/CD, Infrastructure Cloud et Sécurité Applicative

Auteur : Imane HADI

Résumé exécutif

Ce rapport présente une analyse complète de la mise en œuvre d'un pipeline DevSecOps autour de l'application vulnérable SecNotes, développée dans un cadre pédagogique. L'objectif principal est d'intégrer la sécurité à chaque étape du cycle de vie logiciel, en appliquant le principe du Shift Left Security, qui consiste à détecter les vulnérabilités le plus tôt possible dans le processus de développement. Cette approche permet de réduire considérablement les coûts de correction et les risques associés aux failles de sécurité en production.

L'application SecNotes, construite avec Node.js (Express) et MongoDB, a été intentionnellement conçue avec dix vulnérabilités pédagogiques, couvrant un large spectre de catégories allant des injections NoSQL aux secrets codés en dur, en passant par les problèmes de hashage, de validation et de configuration CORS. Ces vulnérabilités ont été détectées et analysées à l'aide de deux outils complémentaires : SonarQube pour l'analyse statique (SAST) et OWASP ZAP pour l'analyse dynamique (DAST).

Le pipeline CI/CD, initialement implémenté sur GitLab CI puis migré vers GitHub Actions, couvre l'ensemble du cycle : installation des dépendances, tests unitaires, analyse statique, construction de l'image Docker, déploiement Kubernetes local, scan dynamique et provisionnement infrastructure AWS via Terraform. L'infrastructure cloud, définie en Infrastructure as Code, comprend un VPC dédié, un cluster AWS EKS et un registre ECR. Ce travail illustre concrètement comment l'approche DevSecOps permet de détecter les problèmes de sécurité tôt dans le cycle de développement, réduisant significativement le coût et le risque des vulnérabilités en production.

1. Présentation du Projet

SecNotes est une application web de type API REST développée avec Node.js (framework Express) et MongoDB, spécifiquement conçue dans un cadre pédagogique DevSecOps. L'application a été intentionnellement développée avec des vulnérabilités afin de servir de cible pour les outils d'analyse de sécurité tels que SonarQube et OWASP ZAP, et d'illustrer les bonnes pratiques de sécurisation d'applications web. Ce choix pédagogique permet aux étudiants et professionnels de comprendre concrètement les mécanismes d'attaque et les stratégies de défense associées.

L'application propose un système de gestion de notes avec authentification JWT, offrant des fonctionnalités CRUD complètes ainsi qu'une fonctionnalité de recherche. Chaque endpoint a été conçu pour exposer des failles spécifiques, permettant une étude systématique des vulnérabilités web courantes. L'environnement est entièrement conteneurisé avec Docker et orchestré via Kubernetes, ce qui facilite le déploiement reproductible et l'intégration dans un pipeline CI/CD automatisé.

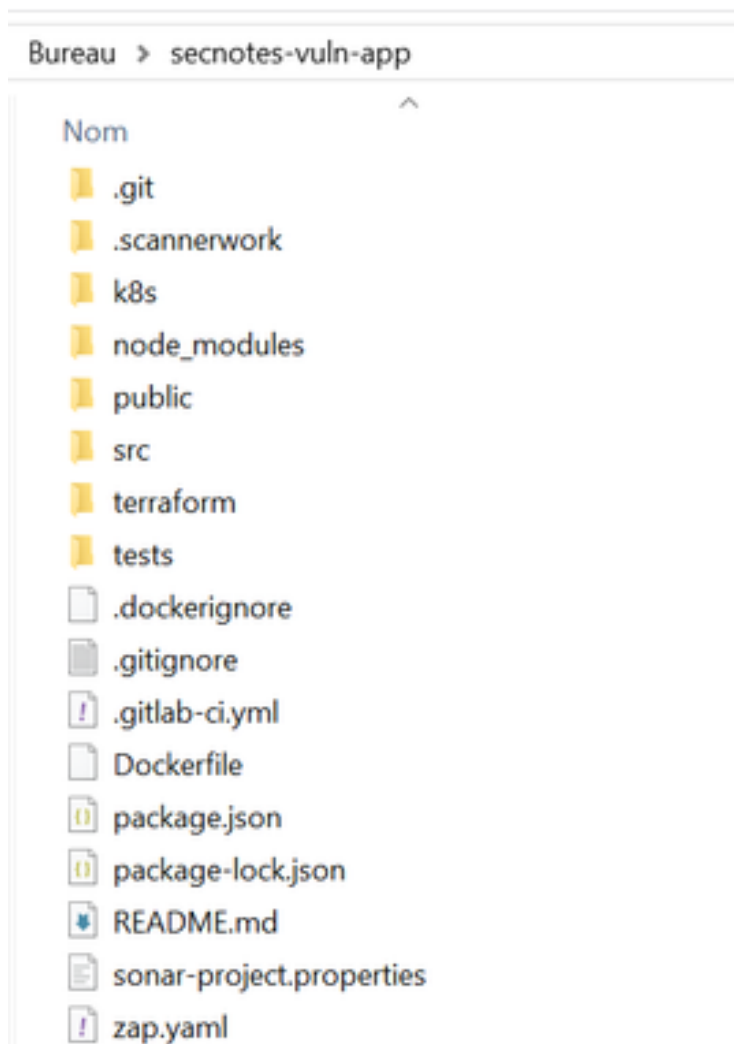


Figure 1 — Vue d'ensemble de l'application SecNotes

1.1 Stack technique

La stack technique de SecNotes a été choisie pour refléter un environnement de développement moderne et représentatif des applications web actuelles. Le backend repose sur Node.js v18+ avec le framework Express, tandis que MongoDB v6 assure le stockage des données. L'authentification est gérée via des tokens JWT (jsonwebtoken). La containerisation Docker et l'orchestration Kubernetes permettent un déploiement cohérent entre les environnements de développement et de production. L'infrastructure cloud est provisionnée sur AWS EKS via Terraform, et le pipeline CI/CD a été migré de GitLab CI vers GitHub Actions.

Composant	Technologie	Version
Backend API	Node.js + Express	v18+
Base de données	MongoDB	v6
Authentification	JWT (jsonwebtoken)	—
Containerisation	Docker	—
Orchestration	Kubernetes (kubectl)	local
Infrastructure cloud	AWS EKS via Terraform	v1.30
CI/CD	GitLab CI → GitHub Actions	—
SAST	SonarQube	local
DAST	OWASP ZAP	stable

Tableau 1 — Stack technique du projet SecNotes

1.2 Endpoints de l’API

L’API SecNotes expose huit endpoints couvrant l’authentification et la gestion complète des notes. Chaque endpoint a été conçu pour présenter des vulnérabilités spécifiques : les endpoints d’authentification exposent des failles d’injection NoSQL et de validation, les endpoints de notes présentent des vulnérabilités IDOR, et l’endpoint de recherche souffre d’un problème d’injection via le parsing de JSON non sanité. Cette conception permet une analyse systématique des différentes catégories de vulnérabilités.

Méthode	Route	Description
POST	/api/auth/register	Inscription utilisateur
POST	/api/auth/login	Connexion et obtention du token JWT
GET	/api/notes	Lister toutes les notes
POST	/api/notes	Créer une note
GET	/api/notes/:id	Obtenir une note par ID
PUT	/api/notes/:id	Modifier une note
DELETE	/api/notes/:id	Supprimer une note
GET	/api/notes/search?q=	Recherche dans les notes

Tableau 2 — Endpoints de l'API SecNotes

2. Architecture DevSecOps

L'architecture DevSecOps mise en place intègre la sécurité à chaque étape du cycle de vie du logiciel, depuis le développement jusqu'au déploiement en production. Cette approche rompt avec le modèle traditionnel où la sécurité est traitée en tant qu'étape finale, souvent sous-traitée à une équipe dédiée. En intégrant les contrôles de sécurité dès les premières phases du développement, l'approche DevSecOps permet de détecter les vulnérabilités lorsque le coût de correction est minimal, réduisant ainsi les risques et les délais de mise en production.

2.1 Vue d'ensemble du pipeline

Le pipeline suit le principe Shift Left Security : les contrôles de sécurité interviennent le plus tôt possible dans le cycle de développement, réduisant ainsi le coût de correction des vulnérabilités. Chaque étape du pipeline est automatisée et produit des artefacts vérifiables, assurant la traçabilité complète entre le code source et l'application déployée. Le pipeline comprend sept étapes principales, de l'installation des dépendances au provisionnement infrastructure, chacune contribuant à la sécurité globale de l'application.

Étape	Outil	Rôle
Install	npm ci	Installation des dépendances
Test	Jest	Tests unitaires et d'intégration
SAST	SonarQube	Analyse statique du code source
Build	Docker	Construction de l'image de conteneur
Deploy	Kubernetes	Déploiement local sur cluster K8s
DAST	OWASP ZAP	Scan dynamique de l'application déployée
IaC	Terraform	Provisionnement infrastructure AWS (manuel)

Tableau 3 — Étapes du pipeline DevSecOps

2.2 Infrastructure cloud AWS

L'infrastructure cloud est provisionnée via Terraform et comprend un ensemble de ressources AWS conçues pour héberger l'application dans un environnement scalable et sécurisé. Le VPC dédié (10.0.0.0/16) est configuré avec deux sous-réseaux publics répartis dans des zones de disponibilité distinctes, assurant une haute disponibilité. L'Internet Gateway et la table de routage permettent l'accès internet nécessaire au fonctionnement du cluster EKS et à la résolution des dépendances externes.

Le cluster AWS EKS, fonctionnant sous Kubernetes v1.30, est configuré avec un groupe de nœuds t3.micro adapté au contexte pédagogique. Les rôles IAM sont définis séparément pour le cluster EKS et les nœuds workers, suivant le principe du moindre privilège. Amazon ECR assure le stockage privé des images Docker construites lors du pipeline CI/CD. Le Security Group, volontairement permissif pour le lab, devra être renforcé pour tout usage en production.

- VPC dédié (10.0.0.0/16) avec 2 sous-réseaux publics en zones de disponibilité distinctes
- Internet Gateway et table de routage pour l'accès internet
- Cluster AWS EKS (Kubernetes v1.30) avec groupe de nœuds t3.micro
- Rôles IAM pour le cluster EKS et les nœuds workers
- Amazon ECR pour le stockage des images Docker
- Security Group (volontairement permissif pour le lab)

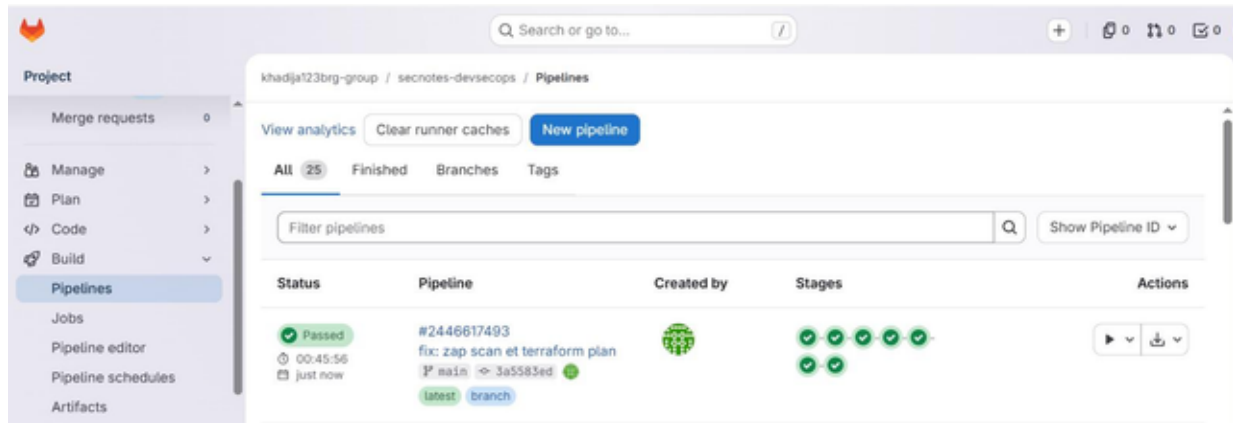


Figure 3 — Architecture DevSecOps et infrastructure cloud

3. Explication des Stages CI/CD

Le pipeline CI/CD est le cœur de l'approche DevSecOps. Chaque stage est conçu pour remplir une fonction spécifique tout en contribuant à la sécurité globale de l'application. L'automatisation complète du pipeline garantit que chaque commit déclenche un ensemble cohérent de vérifications, éliminant les risques liés aux processus manuels. Cette section détaille le rôle, le fonctionnement et les spécificités de chaque étape du pipeline.

3.1 Stage : install

Le stage install exécute la commande `npm ci` pour installer les dépendances de manière déterministe depuis le fichier `package-lock.json`. Contrairement à `npm install`, `npm ci` garantit une installation reproductible en respectant strictement les versions verrouillées dans le fichier de lock, évitant ainsi les régressions involontaires liées à des mises à jour de dépendances. Les `node_modules` sont sauvegardés en artifact pour être réutilisés par les stages suivants, évitant ainsi des téléchargements redondants et réduisant le temps d'exécution global du pipeline.

3.2 Stage : test

Le stage test lance les tests unitaires avec le framework Jest. Dans le contexte de ce lab, les tests sont désactivés car `MongoMemoryServer` présente des incompatibilités avec l'environnement CI, notamment en raison de la nécessité de télécharger un binaire MongoDB qui peut échouer dans certains environnements conteneurisés. Le stage est configuré avec `allow_failure: true` pour ne pas bloquer le pipeline en cas d'échec des tests. En production, ce stage devrait être obligatoire et les tests devraient être adaptés pour fonctionner dans l'environnement CI, par exemple en utilisant des mocks ou une base de données de test légère.

3.3 Stage : sast (Analyse statique)

Le stage SAST constitue l'une des pierres angulaires du pipeline DevSecOps. Sonar-scanner analyse le code source de l'application et envoie les résultats au serveur SonarQube local. Cette étape détecte les vulnérabilités, les code smells et les mauvaises pratiques de sécurité directement dans le code sans exécuter l'application. L'analyse statique permet d'identifier les problèmes tels que les secrets codés en dur, les injections potentielles, les faiblesses cryptographiques et les configurations de sécurité défectueuses avant même que l'application ne soit construite ou déployée, réduisant considérablement le coût de remédiation.

3.4 Stage : build

Le stage build utilise Docker pour construire l'image de l'application et la taguer avec le SHA court du commit Git (`IMAGE_TAG = $CI_COMMIT_SHORT_SHA`). Cette stratégie de tagging garantit une traçabilité complète entre le code source et l'image déployée : à tout moment, il est possible d'identifier précisément quel commit a généré une image donnée. Cette traçabilité est essentielle pour les audits de sécurité et les procédures de rollback en cas de découverte de vulnérabilité en production.

3.5 Stage : deploy_local

Le stage `deploy_local` exécute la commande `kubectl apply -f k8s/` pour déployer l'ensemble des manifestes Kubernetes sur le cluster local. Ces manifestes comprennent le Namespace, la ConfigMap, le Secret, le Deployment et le Service nécessaires au fonctionnement de l'application. Ce stage ne s'exécute que sur la branche main, garantissant que seuls les codes validés par les merge requests sont déployés. L'utilisation de Kubernetes comme plateforme de déploiement offre une isolation des conteneurs, une gestion simplifiée du cycle de vie et une facilité de migration vers un cluster cloud comme AWS EKS.

3.6 Stage : scan (DAST)

Le stage scan constitue la phase d'analyse dynamique (DAST) du pipeline. OWASP ZAP effectue un scan baseline de l'application déployée sur `http://localhost:30080`. Contrairement à l'analyse statique qui examine le code source, le scanner ZAP envoie des requêtes HTTP réelles à l'application et analyse les réponses pour détecter les vulnérabilités courantes telles que les injections, les failles XSS, les en-têtes de sécurité manquants et les problèmes de configuration. Un rapport HTML détaillé est généré en artifact, permettant une analyse approfondie des résultats par l'équipe de sécurité.

3.7 Stage : terraform_plan

Le stage `terraform_plan` est déclenché manuellement (when: manual) uniquement sur la branche main. Terraform initialise le provider AWS, valide la configuration et génère un plan d'exécution sans appliquer les changements. Le fichier `tfplan` est sauvegardé en artifact pour review, permettant à l'équipe infrastructure de valider les modifications avant leur application. Ce mécanisme de gate manuel est une bonne pratique de sécurité pour les modifications d'infrastructure, évitant les déploiements accidentels de ressources cloud qui pourraient avoir des conséquences financières ou de sécurité significatives.

4. Outils Utilisés

L'écosystème d'outils déployé dans ce projet couvre l'ensemble du spectre DevSecOps, de l'automatisation du pipeline à l'analyse de sécurité en passant par la containerisation et le provisionnement infrastructure. Chaque outil a été sélectionné pour sa complémentarité avec les autres et sa capacité à s'intégrer dans un pipeline automatisé. L'utilisation conjointe de SonarQube (SAST) et OWASP ZAP (DAST) offre une couverture de sécurité complète, combinant la détection de vulnérabilités dans le code source et la détection de failles dans l'application en cours d'exécution.

Outil	Catégorie	Description
GitLab CI / GitHub Actions	CI/CD	Automatisation du pipeline de build, test et déploiement
SonarQube	SAST	Analyse statique du code — détection de bugs, vulnérabilités et code smells
OWASP ZAP	DAST	Scan dynamique — simulation d'attaques sur l'application déployée
Docker	Containerisation	Packaging de l'application en image portable et reproductible
Kubernetes (kubectl)	Orchestration	Déploiement et gestion du cycle de vie des conteneurs
Terraform	IaC	Provisionnement déclaratif de l'infrastructure AWS
AWS EKS	Cloud	Cluster Kubernetes managé sur Amazon Web Services
Amazon ECR	Registry	Registre privé Docker hébergé sur AWS
npm / Jest	Dev/Test	Gestion des dépendances et tests automatisés

Tableau 4 — Outils utilisés dans le pipeline DevSecOps

5. Analyse des Vulnérabilités Identifiées

L'application SecNotes a été conçue avec dix vulnérabilités pédagogiques intentionnelles, couvrant un large spectre de catégories OWASP. Ces vulnérabilités ne sont pas le fruit d'un manque de compétence mais d'un choix délibéré pour illustrer les failles les plus courantes dans les applications web modernes. Chaque vulnérabilité a été classée selon sa criticité (Critique, Haute, Moyenne) en suivant les standards CVSS et les

recommandations OWASP Top 10. L'analyse qui suit détaille chaque vulnérabilité, son mécanisme, son impact potentiel et la recommandation de correction associée.

ID	Catégorie	Localisation	Description	Criticité
V1	NoSQL Injection	authController.js, noteController.js	Login accepte des objets MongoDB directs. Recherche parse du JSON sans sanitisation.	Critique
V2	Secret en dur	middleware/auth.js	Clé secrète JWT hardcodée : ma-cle-secrete-en-dur-pour-le-lab	Critique
V3	Absence de validation	authController.js, noteController.js	Aucune validation des entrées. Rôle admin assignable librement.	Haute
V4	IDOR	noteController.js	Aucune vérification d'appartenance des notes. Accès non autorisé.	Haute
V5	Dépendances vulnérables	package.json	Librairies obsolètes avec CVE connues (md5, express ancien).	Moyenne
V6	Hashage faible	authController.js	MD5 pour les mots de passe, obsolète et cassable par rainbow tables.	Haute
V7	Dockerfile non sécurisé	Dockerfile	Exécution en root, image non épinglée, .dockerignore incomplet.	Moyenne
V8	Logs verbeux	Tous les controllers	Corps de requêtes loggés, stack traces	Moyenne

			exposées dans les réponses.	
V9	Secrets K8s en clair	k8s/secret.yaml	Secrets encodés base64 uniquement, non chiffrés.	Haute
V10	CORS permissif	app.js	CORS avec origin: *, permettant tout domaine.	Moyenne

Tableau 5 — Synthèse des vulnérabilités identifiées

5.1 Recommandations de correction

Pour chaque vulnérabilité identifiée, une recommandation de correction spécifique et actionnable a été formulée. Ces recommandations suivent les bonnes pratiques de l'industrie et les standards OWASP. Il est important de noter que la priorité de correction doit suivre la criticité des vulnérabilités : les vulnérabilités critiques (V1, V2) doivent être corrigées en premier, suivies des vulnérabilités haute (V3, V4, V6, V9), puis des vulnérabilités moyennes (V5, V7, V8, V10).

- **V1** — Utiliser `mongoose-sanitize` ou `express-mongo-sanitize` pour nettoyer les inputs avant les requêtes DB. Valider le type de chaque paramètre d'entrée avant de le passer à MongoDB.
- **V2** — Stocker les secrets dans des variables d'environnement ou un gestionnaire de secrets (AWS Secrets Manager, HashiCorp Vault). Ne jamais coder de secrets en dur dans le code source.
- **V3** — Implémenter une validation avec `Joi` ou `express-validator`. Interdire le choix de rôle à l'inscription et définir un schéma strict pour chaque entrée utilisateur.
- **V4** — Ajouter une vérification `note.author.toString() === req.user.id` avant chaque opération sur les notes. Implémenter un contrôle d'accès basé sur les rôles (RBAC).
- **V5** — Mettre à jour régulièrement les dépendances et activer `Dependabot` ou `npm audit` en CI. Supprimer les dépendances inutiles comme `md5`.
- **V6** — Remplacer MD5 par `bcrypt` avec un facteur de coût approprié (minimum 12). `Argon2id` est également recommandé pour les nouvelles implémentations.
- **V7** — Utiliser un utilisateur non-root dans le `Dockerfile`, épingler les versions d'images avec des SHA, et compléter le `.dockerignore` pour exclure les fichiers sensibles.
- **V8** — Implémenter un logger structuré (Winston/Pino) avec masquage des données sensibles. Ne jamais logger les mots de passe ou tokens.
- **V9** — Utiliser des solutions de chiffrement des secrets Kubernetes (Sealed Secrets, AWS KMS, External Secrets Operator). Ne jamais stocker de secrets en clair dans les manifestes.
- **V10** — Configurer CORS avec une liste blanche des origines autorisées explicites. Ne jamais utiliser `origin: *` en production.

6. Analyse des Résultats des Scans

Les résultats des analyses SAST et DAST fournissent une vue complémentaire des vulnérabilités de l'application. L'analyse statique (SonarQube) identifie les problèmes dans le code source, tandis que l'analyse dynamique (OWASP ZAP) détecte les vulnérabilités observables à l'exécution. La combinaison de ces deux

approches offre une couverture de sécurité nettement supérieure à l'utilisation d'un seul type d'analyse, car certaines vulnérabilités ne sont détectables que par l'une ou l'autre méthode.

6.1 SonarQube — Analyse SAST

SonarQube a analysé l'ensemble du code source JavaScript de l'application SecNotes. L'analyse a identifié plusieurs catégories de findings, allant des security hotspots aux code smells en passant par les vulnérabilités confirmées. Les security hotspots, comme les credentials hardcodés et la mauvaise configuration CORS, requièrent une revue manuelle pour confirmer leur exploitabilité. Les vulnérabilités confirmées, telles que le risque d'injection NoSQL et la cryptographie faible, nécessitent une correction immédiate. Les code smells, bien que moins critiques, indiquent des problèmes de qualité du code qui pourraient faciliter l'introduction de vulnérabilités à l'avenir.

Type de finding	Règle SonarQube	Fichier concerné
Security Hotspot	Hardcoded credentials	middleware/auth.js
Bug / Vulnerability	NoSQL injection risk	authController.js
Code Smell	Sensitive data in logs	Tous les controllers
Vulnerability	Weak cryptography (MD5)	authController.js
Security Hotspot	CORS misconfiguration	app.js
Code Smell	Error stack trace exposed	app.js, controllers

Tableau 6 — Résultats de l'analyse SonarQube (SAST)

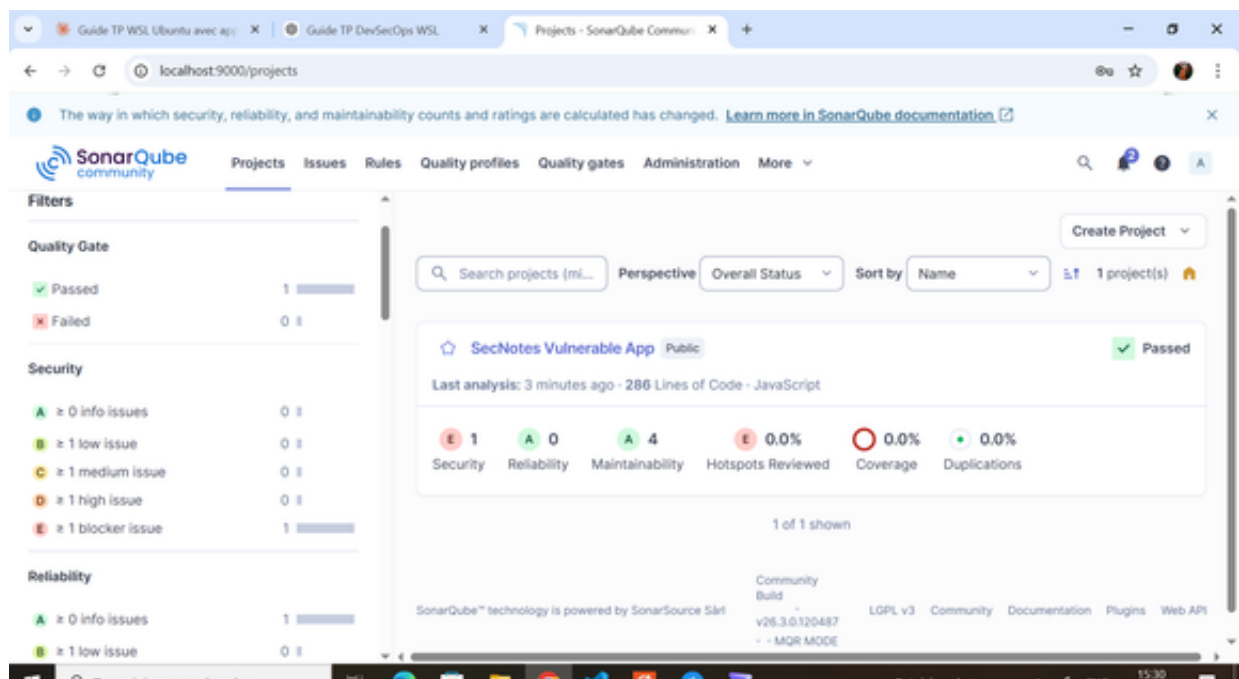


Figure 4 — Résultats de l'analyse SonarQube

6.2 OWASP ZAP — Analyse DAST

Le scan ZAP baseline a été exécuté contre l'application déployée sur <http://localhost:30080>. Le scan passif, sans attaque active, analyse les en-têtes HTTP, les cookies et les réponses pour détecter les mauvaises configurations de sécurité. Cinq alertes ont été identifiées, allant du niveau Medium au niveau Informationnel. L'absence d'en-têtes de sécurité fondamentaux (X-Frame-Options, X-Content-Type-Options) et l'exposition d'informations techniques dans les réponses HTTP constituent les findings les plus significatifs. Le rapport HTML complet généré par ZAP (zap-report.html) est disponible en artifact du pipeline CI/CD pour une analyse détaillée.

Alerte ZAP	Niveau	Description
Missing Anti-clickjacking Header	Medium	Absence de l'en-tête X-Frame-Options
X-Content-Type-Options Header Missing	Low	En-tête anti-MIME sniffing absent
Server Leaks Information via HTTP	Low	Le header Server expose la version d'Express
Application Error Disclosure	Medium	Stack traces exposées dans les réponses d'erreur
Timestamp Disclosure	Informational	Timestamps dans les réponses JSON

Tableau 7 — Résultats du scan OWASP ZAP (DAST)



Figure 5 — Résultats du scan OWASP ZAP

7. Migration GitLab CI vers GitHub Actions

Dans le cadre de ce projet, le pipeline CI/CD a été migré de GitLab CI vers GitHub Actions. Cette migration a nécessité la compréhension des différences conceptuelles entre les deux plateformes et la traduction de chaque

concept GitLab CI vers son équivalent GitHub Actions. La migration a été réalisée avec succès, préservant l'ensemble des fonctionnalités du pipeline original tout en profitant des avantages spécifiques de GitHub Actions, tels que l'écosystème de marketplace d'actions et l'intégration native avec les pull requests GitHub.

GitLab CI	GitHub Actions	Notes
stages	jobs	Découpage du pipeline en étapes
script	run	Commandes shell à exécuter
image (Docker)	container / runs-on	Environnement d'exécution
artifacts	actions/upload-artifact	Sauvegarde des fichiers entre jobs
variables	secrets / env	Variables et secrets CI/CD
only/except	on (push, pull_request)	Conditions de déclenchement
allow_failure	continue-on-error	Jobs non bloquants
tags: [local]	runs-on: self-hosted	Runner auto-hébergé
when: manual	environment + approval	Déclenchement manuel

Tableau 8 — Correspondance GitLab CI / GitHub Actions

7.1 Étapes de migration réalisées

La migration a été réalisée en cinq étapes principales, chacune nécessitant une attention particulière pour garantir la parité fonctionnelle entre les deux plateformes. L'import du repository via l'outil natif de GitHub a permis de conserver l'historique des commits. La création du fichier `.github/workflows/pipeline.yml` a exigé la traduction de chaque stage GitLab en job GitHub Actions, en respectant les dépendances et les conditions d'exécution. La migration des variables CI/CD en GitHub Secrets a impliqué la recreation de six secrets couvrant les clés AWS, le token SonarQube et l'URL du scan DAST.

- **Étape 1** : Import du repository GitLab vers GitHub via github.com/new/import
- **Étape 2** : Création du fichier `.github/workflows/pipeline.yml` avec conversion des stages

- **Étape 3** : Migration des variables CI/CD en GitHub Secrets (6 secrets : AWS keys, SonarQube, DAST URL)
- **Étape 4** : Installation et configuration du GitHub Actions self-hosted runner sur Windows
- **Étape 5** : Validation du runner connecté et en écoute de jobs

8. Conclusion et Perspectives

Ce projet DevSecOps autour de l'application SecNotes a permis de mettre en pratique l'intégration de la sécurité à chaque étape du cycle de développement logiciel. L'approche Shift Left Security, appliquée tout au long du pipeline, a démontré son efficacité dans la détection précoce des vulnérabilités et la réduction des risques associés. Le pipeline CI/CD complet, couvrant l'installation, les tests, l'analyse statique, la containerisation, le déploiement et le scan dynamique, offre une base solide pour tout projet DevSecOps.

8.1 Bilan

Le bilan de ce projet est globalement positif. Un pipeline CI/CD complet a été mis en place, couvrant l'installation, les tests, l'analyse statique (SAST), la containerisation, le déploiement et le scan dynamique (DAST). Dix vulnérabilités pédagogiques ont été identifiées, analysées et documentées avec leurs recommandations de correction, couvrant les catégories OWASP les plus critiques. L'infrastructure cloud AWS (VPC, EKS, ECR) a été définie en Infrastructure as Code via Terraform, permettant un provisionnement reproductible et versionné. La migration complète de GitLab CI vers GitHub Actions a été réalisée avec succès, préservant toutes les fonctionnalités du pipeline original.

8.2 Points d'amélioration

Plusieurs axes d'amélioration ont été identifiés pour renforcer davantage la posture de sécurité du pipeline et de l'application. Ces améliorations concernent aussi bien les outils d'analyse que les pratiques de déploiement et de gestion des secrets. Leur implémentation permettrait d'atteindre un niveau de maturité DevSecOps supérieur et de mieux préparer l'application pour un éventuel passage en production.

- **Scan DAST actif** : Activer le mode attack actif de ZAP pour des scans DAST plus complets, incluant les tests d'injection et de XSS actifs.
- **Quality Gate SonarQube** : Mettre en place un Quality Gate SonarQube bloquant le pipeline en cas de nouvelles vulnérabilités, empêchant ainsi le déploiement de code non conforme.
- **Scan d'images Docker** : Implémenter Trivy ou Gype pour le scan des images Docker (SCA), détectant les vulnérabilités dans les paquets système et les dépendances de l'image.
- **Chiffrement des secrets K8s** : Chiffrer les secrets Kubernetes avec Sealed Secrets ou AWS KMS, éliminant le risque de lecture par tout utilisateur ayant accès au cluster.
- **Tests d'intégration** : Ajouter des tests d'intégration réels avec une base MongoDB de test, complétant les tests unitaires et améliorant la couverture de test globale.